

Lab 39: Model Evaluation Metrics

Aim

To compute and interpret standard **classification evaluation metrics** such as the **Confusion Matrix, Accuracy, Precision, Recall, and F1-Score** for a trained model.

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train a Decision Tree classifier.
5. Generate predictions for the test data.
6. Compute the confusion matrix.
7. Calculate accuracy, precision, recall, and F1-score.
8. Display all evaluation metrics.

Program

```
# Lab 39: Model Evaluation Metrics
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
# Load Iris dataset

X, y = load_iris(return_X_y=True)


# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.3,

    random_state=1

)


# Train the model

clf = DecisionTreeClassifier(random_state=1)

clf.fit(X_train, y_train)


# Make predictions

pred = clf.predict(X_test)


# Display evaluation metrics

print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

print("Accuracy:", accuracy_score(y_test, pred))

print("Precision (macro):", precision_score(

    y_test, pred, average="macro"

))

print("Recall (macro):", recall_score(

    y_test, pred, average="macro"

))

print("F1-score (macro):", f1_score(

    y_test, pred, average="macro"

))
```

Sample Output

Confusion Matrix:

```
[[14 0 0]
 [ 0 17 1]
 [ 0 0 13]]
```

Accuracy: 0.9777777777777777

Precision (macro): 0.9761904761904763

Recall (macro): 0.9814814814814815

F1-score (macro): 0.9781305114630964

Result

Thus, the **classification model was successfully evaluated** using the confusion matrix, accuracy, precision, recall, and F1-score.